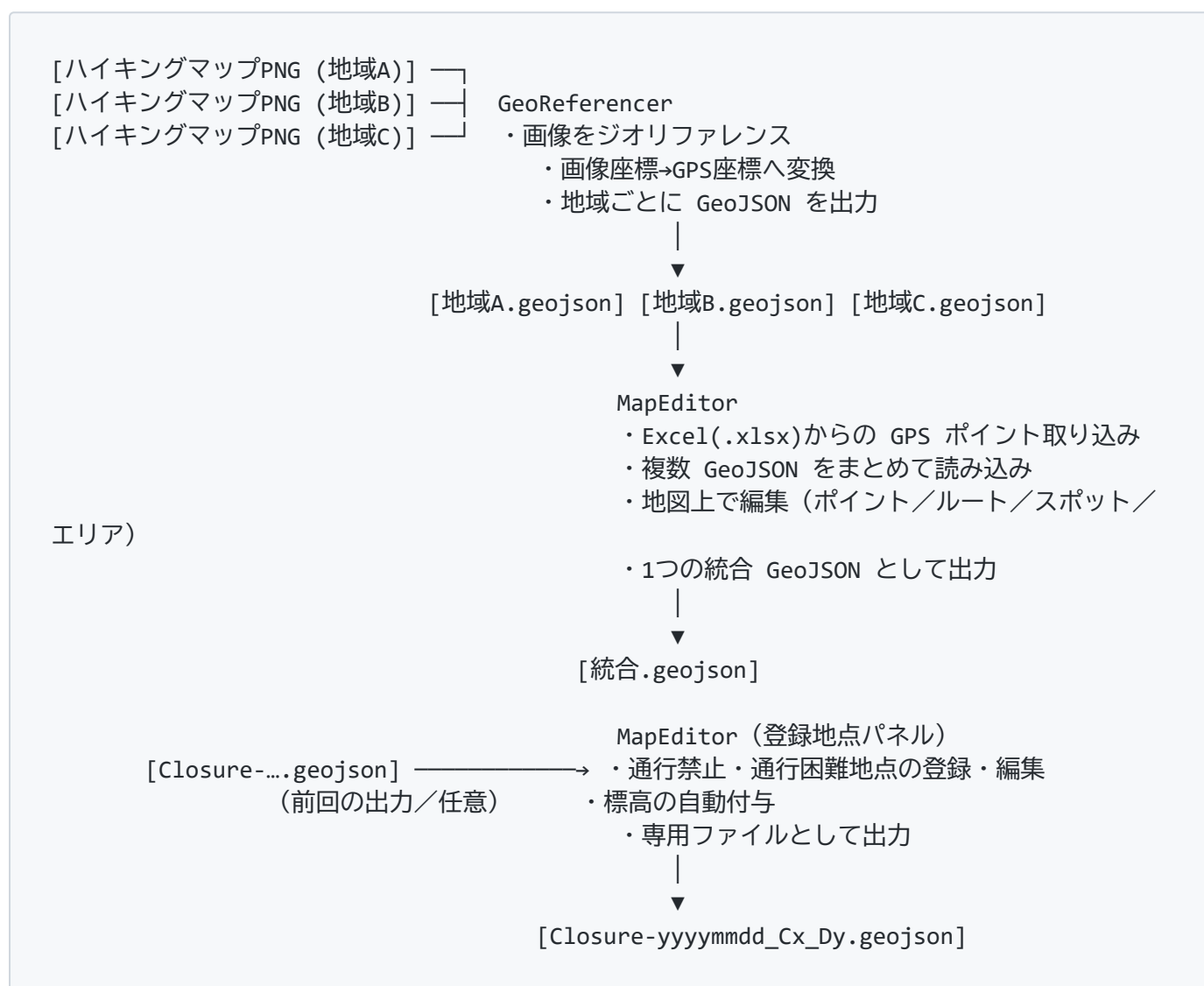


GeoJSON データ仕様書

1. 概要

本ドキュメントは、**GeoReferencer** からエクスポートされ **MapEditor** に取り込まれる GeoJSON ファイル、MapEditor が出力する統合 GeoJSON ファイル、および MapEditor が入出力する**通行禁止・通行困難地点ファイル**のフォーマットを定義します。

1.1 データフロー



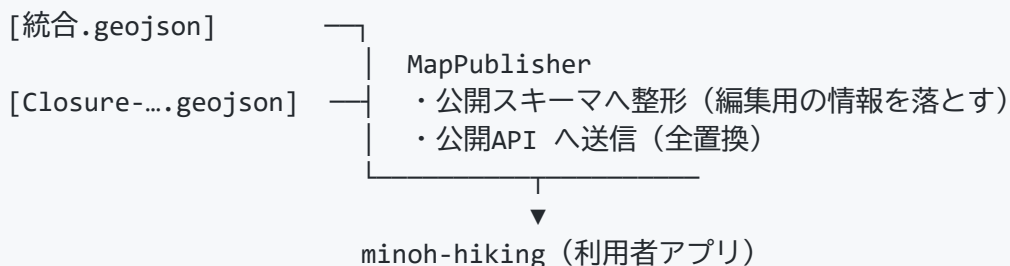
- **GeoReferencer**: PNG 画像を国土地理院地図に対してジオリファレンス（最小二乗法による 6 パラメータアフィン変換）し、画像内座標を GPS 座標（緯度・経度）に変換した結果を GeoJSON で出力します。地域ごとに 1 ファイルを出力する運用を想定します。
- **MapEditor**: GeoReferencer が地域ごとに出力した複数 GeoJSON をまとめて読み込み、地図上で編集後、1 つの GeoJSON として再出力します。

通行禁止・通行困難地点（closure）について: closure データは統合 GeoJSON には**含まれません**（読み込み時も無視されます）。MapEditor の登録地点パネルで編集し、専用ファイルとして個別に入出力します（5 章）。

1.2 公開との関係（MapPublisher）

MapEditor が出力するファイルは**作業用**であり、そのまま利用者へ配信されるわけではありません。

公開は **MapPublisher** が行い、送信前に**公開スキーマへ整形**します。



整形時に落とされる主なものは次のとおりです。

type	落とされるプロパティ	理由
spot	id / source / description	表示に使わない。ルートはスポットを name で参照するため id は参照されない
ポイント GPS	pointId	id と同値のため
route	startPoint / endPoint	端点は startPointGPS / endPointGPS の座標で渡すため、ID 参照は不要

また point / route_waypoint / area は**公開されません**（利用者アプリで描画しないため）。

MapEditor 側は出力形式を変更しません。 編集に必要な識別子は作業ファイルに残したままとし、配信データとの差は MapPublisher が吸収します。編集の都合と配信の都合を分離することで、どちらか一方の事情がもう一方に波及しないようにしています。

公開スキーマの正本は minoh-hiking の docs/publish-api-202608.md §3（契約バージョン 2.0）です。二重管理を避けるため、本書には書き写しません。

2. 共通仕様

項目	値
フォーマット	GeoJSON (RFC 7946)
ルートタイプ	FeatureCollection

項目	値
拡張子	<code>.geojson</code>
文字コード	UTF-8
座標系	WGS84 (EPSG:4326)
座標順序	<code>[経度(longitude), 緯度(latitude)]</code> または <code>[経度, 緯度, 標高(メートル)]</code>
標高	取得済みの場合のみ3番目の要素として含む。未取得時は <code>[lng, lat]</code> の2要素

2.1 数値精度

- GeoReferencer 出力: 経度・緯度を **小数点以下5桁** に丸めて出力（標高は小数点以下1桁）。
- MapEditor 出力: 経度・緯度・標高をいずれも **小数点以下5桁** に丸めて出力（内部に保持した座標値は読み込んだ精度を維持し、丸めは出力時のみ適用）。統合 GeoJSON・通行禁止・通行困難地点ファイルとも同じ扱いです。

2.2 FeatureCollection の基本構造

```
{
  "type": "FeatureCollection",
  "features": [ /* Feature の配列 */ ]
}
```

通行禁止・通行困難地点ファイルは、これに加えてトップレベルの `updatedAt`（および任意の `version`）を持ちます（5.2 節）。

3. GeoReferencer 出力ファイル仕様（MapEditor の入力）

GeoReferencer が「GeoJSON 出力」操作で生成するファイルの仕様です。MapEditor はこのフォーマットの複数ファイルを読み込みます。

3.1 ファイル名

```
{略称}-GPS-{P{point数}_R{route数}_S{spot数}_A{area数}}-{YYYYMMDD}.geojson
```

- **略称**: PNG 画像ファイル名の先頭から区切り文字（`-`、`_`、半角スペース、`.`）の前までを使用。
- **件数部**: ポイント／ルート／スポット／エリアの件数。0 件の項目は省略。すべて 0 件の場合は件数部全体を省略。
- **日付**: 出力日 `YYYYMMDD`。

例:

- minoh-GPS-P5_R2_S3-20260427.geojson
- takao-GPS-P10_A1-20260427.geojson

3.2 Feature 種別

`features` 配列には次の 4 種類の Feature が含まれます。

type	Geometry	説明
<code>point</code>	Point	ジオリファレンス基準点／画像内ポイント
<code>route</code>	LineString	画像上のルートを変換した経路
<code>spot</code>	Point	画像上のスポット
<code>area</code>	Polygon	画像上のエリア

3.2.1 ポイント (`type: "point"`)

プロパティ	値・型	説明
<code>id</code>	String	ポイント ID
<code>name</code>	String	ポイント名称
<code>type</code>	<code>"point"</code>	固定値
<code>source</code>	<code>"image_transformed"</code>	固定値
<code>description</code>	<code>"画像ポイント (GPS変換済) "</code>	固定値

```
{
  "type": "Feature",
  "properties": {
    "id": "A-01",
    "name": "登山口",
    "type": "point",
    "source": "image_transformed",
    "description": "画像ポイント (GPS変換済) "
  },
  "geometry": {
    "type": "Point",
    "coordinates": [135.47204, 34.85367, 150.5]
  }
}
```

3.2.2 ルート (`type: "route"`)

中間点 (waypoint) を順番に並べた LineString。

プロパティ	値・型	説明
<code>id</code>	String	ルート ID。形式 <code>route_{startPoint}_to_{endPoint}</code>
<code>name</code>	String	表示名。形式 <code>{startPoint} ～ {endPoint}</code>
<code>type</code>	<code>"route"</code>	固定値
<code>startPoint</code>	String	開始ポイント ID（取得不能時 <code>"unknown_start"</code> ）
<code>endPoint</code>	String	終了ポイント ID（取得不能時 <code>"unknown_end"</code> ）
<code>startPointGPS</code>	<code>[lng, lat]</code> または <code>null</code>	開始ポイントの GPS 座標。同一 GeoJSON 内のポイント／スポット Feature から解決
<code>endPointGPS</code>	<code>[lng, lat]</code> または <code>null</code>	終了ポイントの GPS 座標
<code>source</code>	<code>"image_transformed"</code>	固定値
<code>description</code>	<code>"ルート（GPS変換済）"</code>	固定値

`coordinates` は中間点を順番に並べた `[lng, lat]` または `[lng, lat, elevation]` の配列。

```
{
  "type": "Feature",
  "properties": {
    "id": "route_A-01_to_A-02",
    "name": "A-01 ～ A-02",
    "type": "route",
    "startPoint": "A-01",
    "endPoint": "A-02",
    "startPointGPS": [135.47204, 34.85367, 100.1],
    "endPointGPS": [135.47500, 34.85800, 120.3],
    "source": "image_transformed",
    "description": "ルート（GPS変換済）"
  },
  "geometry": {
    "type": "LineString",
    "coordinates": [
      [135.47210, 34.85400, 100.1],
      [135.47300, 34.85500, 110.2],
      [135.47400, 34.85700, 120.3]
    ]
  }
}
```

`startPointGPS` / `endPointGPS` の解決優先順位:

1. 同一 GeoJSON 内のポイント／スポット Feature から、`id` または `name` の一致でルックアップ。
2. ルート定義中に `startPoint` / `endPoint` の `lat` / `lng` がある場合はそれを使用。
3. いずれも該当しない場合は `null`。

3.2.3 スポット (type: "spot")

プロパティ	値・型	説明
id	String	スポット ID。 spot{連番2桁}_{spotName} 形式（例 spot01_見晴台 ） またはスポット名
name	String	スポット名称
type	"spot"	固定値
source	"image_transformed"	固定値
description	"スポット (GPS変換済) "	固定値

```
{
  "type": "Feature",
  "properties": {
    "id": "spot01_見晴台",
    "name": "見晴台",
    "type": "spot",
    "source": "image_transformed",
    "description": "スポット (GPS変換済) "
  },
  "geometry": {
    "type": "Point",
    "coordinates": [135.47210, 34.85400, 400.9]
  }
}
```

3.2.4 エリア (type: "area")

プロパティ	値・型	説明
id	String	エリア ID（未指定時は area_{タイムスタンプ} を自動付与）
name	String	エリア名称（未指定時は "名称未設定エリア" ）
type	"area"	固定値
source	"image_transformed"	固定値
description	"エリア (GPS変換済) "	固定値

coordinates は GeoJSON Polygon 仕様に従い [[[lng,lat], ..., [lng,lat]]] の 3 重ネスト構造。多角形の閉合（始点と終点を一致させる）は出力時に自動付与されます。各頂点の標高が取得済みの場合は [lng, lat, elevation] として出力。

```
{
  "type": "Feature",
  "properties": {
    "id": "area_01",
    "name": "駐車場エリア",
    "type": "area",
    "source": "image_transformed",
    "description": "エリア（GPS変換済）"
  },
  "geometry": {
    "type": "Polygon",
    "coordinates": [[
      [135.47200, 34.85300, 100.2],
      [135.47210, 34.85300, 100.2],
      [135.47210, 34.85310, 100.3],
      [135.47200, 34.85310, 100.3],
      [135.47200, 34.85300, 100.2]
    ]]
  }
}
```

3.3 出力に関する注意事項

- **重複出力の回避**: 同一データが複数の内部データソースに存在する場合、片方のみが出力されます。
- **スポットの最新採用**: 同一 ID のスポットが複数存在する場合、最新のもののみが出力されます。
- **ルートのグルーピング**: 中間点はルート ID ごとにグループ化され、各グループが 1 つの LineString Feature として出力されます。
- **標高**: 国土地理院標高 API から取得した値が利用可能な場合のみ、3 番目の座標要素として含まれます。

4. MapEditor 出力ファイル仕様（統合 GeoJSON）

MapEditor が「GeoJSON 出力」操作で生成する統合ファイルの仕様です。GeoReferencer 由来の Feature と、Excel から取り込んだ GPS ポイントを 1 つの FeatureCollection としてまとめて出力します。

4.1 ファイル名

MapGPS-{YYYYMMDD}_P{ポイント数}_R{ルート数}_S{スポット数}.geojson

例: MapGPS-20260817_P15_R5_S8.geojson

4.2 Feature 種別

type	Geometry	由来
point	Point	GeoReferencer 等から読み込んだ画像変換ポイント（入力時のプロパティを保持）
ポイントGPS	Point	Excel(xlsx) 読み込みによる GPS ポイント
route	LineString	ルート編集結果（内部の route_waypoint から再構成）
spot	Point	スポット編集結果
area	Polygon	エリア編集結果

注 1: 内部編集用の type: "route_waypoint" Point Feature は出力時に route LineString に集約され、ファイルには出力されません（4.2.6 参照）。

注 2: 通行禁止・通行困難地点（type: "closure"）は統合 GeoJSON には出力されません。専用ファイルとして個別に出力します（5 章）。

4.2.1 ポイント（type: "point"）

GeoReferencer 等の外部ツールでジオリファレンス変換された基準点。MapEditor は入力時の properties をそのまま保持して再出力します。

プロパティ	値・型	説明
type	"point"	固定、必須
id	String	ポイント ID
name	String	ポイント名称
source	String	入力時の値を保持（例: "image_transformed"）
description	String	入力時の値を保持

```
{
  "type": "Feature",
  "properties": {
    "id": "A-01",
    "name": "登山口",
    "type": "point",
    "source": "image_transformed",
    "description": "画像ポイント（GPS変換済）"
  },
  "geometry": {
    "type": "Point",
    "coordinates": [139.123456, 35.654321, 150.5]
  }
}
```


4.2.2 ポイントGPS (`type: "ポイントGPS"`)

Excel(.xlsx) から読み込んだ GPS ポイントデータ。MapEditor 独自の間種別。

プロパティ	値・型	説明
<code>type</code>	<code>"ポイントGPS"</code>	固定、必須
<code>id</code>	String	ポイント ID (Excel の <code>pointId</code> 列と同値)
<code>name</code>	String	ポイント名称
<code>pointId</code>	String	ポイント ID (<code>id</code> と同値)
<code>description</code>	String	備考 (Excel に備考列がある場合)

座標は `[経度, 緯度]` または `[経度, 緯度, 標高]` (Excel に標高列がある場合)。

```
{
  "type": "Feature",
  "properties": {
    "type": "ポイントGPS",
    "id": "G-51",
    "name": "登山口",
    "pointId": "G-001",
    "description": "駐車場あり"
  },
  "geometry": {
    "type": "Point",
    "coordinates": [135.472041, 34.853667, 150.5]
  }
}
```

4.2.3 ルート (`type: "route"`)

ルート編集結果。内部の中間点群を順序通りに並べた LineString として出力。GeoReferencer 出力仕様 (3.2.2 節) と整合する形で、ID と座標を分離して保持します。

プロパティ	値・型	説明
<code>type</code>	<code>"route"</code>	固定、必須
<code>id</code>	String	ルート ID。フォーマット: <code>route_{startPoint}_to_{endPoint}</code> 。 <code>startPoint</code> / <code>endPoint</code> と常に一致
<code>startPoint</code>	String	開始ポイント ID (真の参照情報)
<code>endPoint</code>	String	終了ポイント ID (真の参照情報)
<code>startPointGPS</code>	<code>[lng, lat]</code> / <code>[lng, lat, elevation]</code> / <code>null</code>	開始ポイントの GPS 座標。出力時点での <code>startPoint</code> 解決結果のスナップショット。解決失敗時は <code>null</code> または省略

プロパティ	値・型	説明
endPointGPS	[lng, lat] / [lng, lat, elevation] / null	終了ポイントの GPS 座標。出力時点での endPoint 解決結果のスナップショット。解決失敗時は null または省略

座標は中間点の座標を順番に並べたもの。各座標は [経度, 緯度] または [経度, 緯度, 標高]。

ID と座標の関係（補完ルール）：

- startPoint / endPoint (ID) が 真の参照情報源 です。
- startPointGPS / endPointGPS (座標) は出力時点でのスナップショットであり、参照先ポイントが移動・改名された場合は古くなり得ます。
- 読み込み時の優先順位:
 1. startPoint / endPoint の ID で動的解決を試み、解決できればその 現在座標 を採用（ポイント追従を維持）
 2. ID で解決できない場合のみ、 startPointGPS / endPointGPS の 座標値 にフォールバック
- 出力時の整合保証:
 - id (routeId) に含まれる ID と startPoint / endPoint は常に一致させて出力
 - startPointGPS / endPointGPS は、 startPoint / endPoint から下記解決順序で 引き直した現在座標 で書き出す

開始・終了ポイントの ID 解決順序: startPoint / endPoint の値をキーとして、以下の優先順位で対応する Feature を検索します。

1. type: "ポイントGPS" の id に一致
2. type: "point" の id に一致
3. type: "spot" の id または name に一致（同名スポットが複数ある場合は、相手側端点に最も近いものを採用）
4. 上記すべてで解決できない場合は startPointGPS / endPointGPS の座標値にフォールバック

```
{
  "type": "Feature",
  "properties": {
    "type": "route",
    "id": "route_A-01_to_A-05",
    "startPoint": "A-01",
    "endPoint": "A-05",
    "startPointGPS": [135.472041, 34.853667, 150.5],
    "endPointGPS": [135.474000, 34.855000, 170.0]
  },
  "geometry": {
    "type": "LineString",
    "coordinates": [
      [135.472041, 34.853667, 150.5],
      [135.473000, 34.854000, 160.0],
      [135.474000, 34.855000, 170.0]
    ]
  }
}
```

4.2.4 スポット（type: "spot"）

ポイント以外の地物（見晴台、休憩所等）。

プロパティ	値・型	説明
type	"spot"	固定、必須
name	String	スポット名称。MapEditor 上で新規作成した場合の初期値は "仮{連番}"（例: "仮1"、"仮2"、...）
id	String	スポット ID（入力ファイル由来。MapEditor 新規作成時は付与されない）
description	String	入力ファイル由来のプロパティを保持

座標は [経度, 緯度] または [経度, 緯度, 標高]。

```
{
  "type": "Feature",
  "properties": {
    "type": "spot",
    "name": "見晴台"
  },
  "geometry": {
    "type": "Point",
    "coordinates": [135.480, 34.860, 400]
  }
}
```

4.2.5 エリア（type: "area"）

領域を表すポリゴン（駐車場、休憩エリア等）。

プロパティ	値・型	説明
type	"area"	固定、必須
name	String	エリア名称。MapEditor 上で新規作成した場合の初期値は "エリア{連番}"（例: "エリア1"、"エリア2"、...）
id	String	エリア ID（入力ファイル由来。MapEditor 新規作成時は付与されない）

座標は GeoJSON Polygon 仕様に従い、最初の配列が外周リング、以降が穴を表すリング。各リングの始点と終点の座標は同一であること。

```
{
  "type": "Feature",
  "properties": {
    "type": "area",
    "name": "駐車場エリア"
  },
  "geometry": {
    "type": "Polygon",
    "coordinates": [[
      [135.470, 34.850, 100.2],
      [135.471, 34.850, 100.2],
      [135.471, 34.851, 100.3],
      [135.470, 34.851, 100.3],
      [135.470, 34.850, 100.2]
    ]]
  }
}
```

4.2.6 内部表現: `route_waypoint` (ファイル出力なし)

MapEditor 内部での編集で、各ルート of 全座標を Point Feature として保持しています。エクスポートファイルには出力されません。

プロパティ	値・型	説明
<code>type</code>	<code>"route_waypoint"</code>	固定
<code>route_id</code>	String	所属するルート ID (例: <code>route_A-01_to_A-05</code>)
<code>waypoint_number</code>	String	中間点番号 (<code>"1"</code> から開始)

5. 通行禁止・通行困難地点ファイル仕様 (MapEditor 専用入出力)

MapEditor の「登録地点」パネルが読み書きする専用ファイルの仕様です。統合 GeoJSON とは独立したファイルとして扱います。

5.1 ファイル名

Closure-{YYYYMMDD}_C{通行止め件数}_D{通行困難件数}.geojson

例: `Closure-20260817_C3_D2.geojson`

- `C`: 区分が `closed` (通行止め) の件数
- `D`: 区分が `difficult` (通行困難) の件数
- 区分は 2 種類のため、`C` と `D` の合計が登録地点の総数と一致します

5.2 FeatureCollection の構造

```
{
  "type": "FeatureCollection",
  "version": "2026-08-17.1",
  "updatedAt": "2026-08-17T10:30:00+09:00",
  "features": [ /* closure Feature の配列 */ ]
}
```

プロパティ	値・型	必須	説明
type	"FeatureCollection"	○	固定
version	String		公開用のデータバージョン。MapEditor には編集欄がなく、読み込んだファイルに値があった場合のみ引き継いで出力します
updatedAt	String (ISO 8601)	○	ファイル出力日時。タイムゾーンオフセット付き
features	Array	○	closure Feature の配列

5.3 Feature 仕様

5.3.1 プロパティ

プロパティ	値・型	必須	説明
type	"closure"	○	固定値。通行禁止・通行困難地点専用ファイルであることの目印
id	String	○	地点 ID。C- {連番2桁} 形式（例 C-01）。全地点で一意。MapEditor が自動採番
name	String	○	地点名称。新規作成時の初期値は 地点{連番}
kind	"closed" / "difficult"	○	区分。未選択の状態は存在せず、必ずいずれかを出力（新規登録時の既定値は "closed"）
reason	String		登録理由。値があるときのみ出力。標準値は 工事 / 倒木 / 落石 （新規登録時の既定値は 工事 。画面上の「その他」は値なしとして扱う）
note	String		備考。値があるときのみ出力
relatedRoute	String		関連ルート ID。値があるときのみ出力（入力に含まれていれば保持して再出力。UI からの設定手段はなし）
updatedAt	String (YYYY-MM-DD)	○	地点の最終更新日（追加・移動・属性編集・標高付与で更新）

プロパティの出力順: type → id → name → kind → (reason) → (note) → (relatedRoute) → updatedAt

5.3.2 kind (区分) の値

値	意味	MapEditor での表示
closed	通行止め	×印 (赤 #DC2626)
difficult	通行困難	三角形 (橙 #F59E0B)

旧仕様の "unknown" (未選択) は廃止しました。読み込んだファイルに "unknown" が含まれていた場合は "closed" (通行止め) として取り込みます。

5.3.3 Geometry

- type: "Point"
- coordinates: [経度, 緯度] または [経度, 緯度, 標高]
- 標高は国土地理院標高 API から取得できた場合のみ 3 番目の要素として含みます。

5.3.4 出力例

```
{
  "type": "Feature",
  "properties": {
    "type": "closure",
    "id": "C-01",
    "name": "崩落地点",
    "kind": "closed",
    "reason": "落石",
    "note": "迂回路あり",
    "updatedAt": "2026-08-17"
  },
  "geometry": {
    "type": "Point",
    "coordinates": [135.47204, 34.85367, 150.5]
  }
}
```

5.4 読み込み時の判定と正規化

複数ファイルを同時に選択でき、すべての Feature を合算して取り込みます。

取り込み対象の判定 (すべて満たすもの) :

- Geometry が Point であり、coordinates が配列であること
- properties.type が未設定、または "closure" であること
- properties.type が統合 GeoJSON 由来の既知種別 (ポイントGPS、point、route、route_waypoint、spot、スポット、area) でないこと

`properties.type` が無い Feature も取り込む仕様です（公開ストア由来の GeoJSON には `type` が含まれないため）。統合 GeoJSON を誤って選択した場合は、条件 3 により既知種別が除外されます。

正規化処理:

対象	処理
<code>type</code>	"closure" を設定
<code>id</code>	未設定なら <code>C-{連番2桁}</code> 形式で自動採番（既存の最大番号 +1）
<code>kind</code>	<code>closed</code> / <code>difficult</code> 以外の値（ <code>unknown</code> 、未設定等）は <code>closed</code> （通行止め）に正規化
<code>status</code>	廃止済みプロパティのため読み込み時に除去
ID 重複	既存地点と同一 <code>id</code> の地点はスキップ（件数を通知）
<code>version</code> (トップレベル)	値があれば保持し、出力時に引き継ぐ

6. MapEditor の読み込み処理仕様（ファイル単位の振る舞い）

GeoJSON 読み込み時、MapEditor はファイルレベルで次の処理を行います。

- 1. **複数ファイルの一括選択:** 複数の `.geojson` ファイルを同時に選択でき、全ファイルの Feature を 1 つに統合します。
- 2. **読み込み種別選択モдал:** 全ファイルの合計件数（ポイント／ルート／スポット／エリア）を表示し、種別ごとに読み込み対象を選択できます。
- 3. **同一 ID 重複の除外:** `type: "point"` および `type: "ポイントGPS"` について、既存データと同一 `id` を持つ Feature は新規追加分をスキップします。バッチ内の重複も除外されます。
- 4. **`type: "route"` LineString の自動展開:** `id` が `route_{X}_to_{Y}` パターンに一致するルートは、内部編集用に `route_waypoint` Point Feature へ自動展開されます。各座標が順番に `waypoint_number` 1, 2, 3, ... として登録されます。
- 5. **編集対象としての登録:** `route_waypoint` Point は `route_id` ごとにグループ化し `waypoint_number` 順に並べて編集対象として登録します。
- 6. **closure の除外:** `type: "closure"` の Point Feature は取り込みません（登録地点パネルの専用読み込みで扱います）。

6.1 Excel(.xlsx) 読み込み

`.xlsx` ファイルを選択した場合は、各行を `type: "ポイントGPS"` の Feature に変換して取り込みます。同一 `id` の既存ポイントGPS が存在する場合は Excel の内容で上書きされます。

Excel 列	GeoJSON プロパティ
ポイントID	<code>id</code> 、 <code>pointId</code>

Excel 列	GeoJSON プロパティ
名称	<code>name</code>
緯度	<code>coordinates[1]</code>
経度	<code>coordinates[0]</code>
標高（任意）	<code>coordinates[2]</code>
備考（任意）	<code>description</code>

7. MapEditor の出力処理仕様（ファイル単位の振る舞い）

統合 GeoJSON のエクスポート時、MapEditor は内部状態に対して次の変換を行います。

1. `type: "route_waypoint"` **Point** をすべて除外する。
2. `type: "route"` **LineString** をすべて除外する（再生成のため）。
3. 除外した `route_waypoint` Point を `route_id` でグループ化し、`waypoint_number` 昇順に並べて LineString へ集約する。
4. 集約された LineString を `type: "route"` Feature として追加する。プロパティは以下の通り付与する：
 - `type` / `id` を付与（`id` は `route_{startPoint}_to_{endPoint}` 形式）
 - `startPoint` / `endPoint` (ID) を `id` (routeId) から抽出して付与
 - `startPointGPS` / `endPointGPS` (座標) を、`startPoint` / `endPoint` の ID から [4.2.3 ID 解決順序](#) に従って引き直した現在座標で付与（解決失敗時は `null` または省略）
5. その他の Feature（`point`、`ポイントGPS`、`spot`、`area`）はそのまま保持する。

通行禁止・通行困難地点は統合 GeoJSON とは別に保持しているため、上記の処理対象外です。登録地点パネルの「ファイル出力」で、[5 章](#) の仕様に従って個別に出力します。

8. 補足

- `source` / `description` フィールド: 入力 GeoJSON に含まれていれば保持して再出力されますが、MapEditor が新規生成するルート（`route`）、スポット（`spot`）、エリア（`area`）の Feature には付与されません。
- `name` フィールド: ルート（`route`）Feature にはエクスポート時に付与されません（`startPoint` / `endPoint` で識別）。
- **ポイントGPS** は MapEditor 独自の間種別であり、GeoReferencer 標準仕様には含まれません。Excel 読み込み機能の入力データとして使用されます。
- **通行禁止・通行困難地点** は統合 GeoJSON に混在させないでください。混在したファイルを統合 GeoJSON として読み込んだ場合、closure Feature は取り込まれずに失われます（元ファイルは変更されません）。

9. 関連ドキュメント

- 入力 JSON 仕様: `dataspec-json-202604.md`

- Firebase DB 仕様: [firebase-dbspec-202512.md](#)
- 機能仕様: [funcspec-202608.md](#)
- 利用者の手引: [UsersGuide-202608.md](#)
- 公開API・公開スキーマ（正本）: minoh-hiking [docs/publish-api-202608.md](#)
- 公開ツール: MapPublisher [docs/funcspec-202608.md](#) / [docs/dataspec-202608.md](#)
- 前バージョン: [dataspec-geojson-202607.md](#)

作成日: 2026 年 8 月 17 日

バージョン: 2.9（公開との関係を追記）

変更履歴:

- v2.9 (2026-08-20): 公開との関係（1.2 節）を追加。MapEditor の出力は作業用ファイルであり、公開時に MapPublisher が公開スキーマへ整形すること、その際に落とされるプロパティと公開されない type を明記。

MapEditor 側の出力形式は変更なし。

- v2.8 (2026-08-17): 通行禁止・通行困難地点の `kind` から `"unknown"`（未選択）を廃止し、`"closed"` / `"difficult"` の 2 値に変更。新規登録時の既定値（`kind`: `"closed"`、`reason`: `"工事"`）と、読み込み時に不正な `kind` を `"closed"` へ正規化する仕様を追記。

- v2.7 (2026-08-17): 通行禁止・通行困難地点（closure）の登録機能が MapEditor に追加されたことに伴い、専用ファイルの仕様（ファイル名・FeatureCollection 構造・Feature プロパティ・読み込み時の判定と正規化）を 5 章として追加し、以降の章番号を繰り下げ。データフロー図・読み込み／出力処理仕様に closure の扱いを追記。

- v2.6 (2026-07-28): 通行止め・通行困難場所（closure）の編集・入出力機能が MapEditor から別アプリケーション

ClosureEditor へ移行したことに伴い、MapEditor 側ドキュメント [dataspec-geojson-closure-202606.md](#) への参照を削除。

- v2.5 (2026-06-21): 通行止め・通行困難場所（closure）の GeoJSON 入出力仕様を独立ドキュメント [dataspec-geojson-closure-202606.md](#) として分離。本ドキュメントが扱う統合 GeoJSON には closure を含めないことを概要に明記。

- v2.4 (2026-04-28): MapEditor の `route` Feature に `startPoint` / `endPoint`（ID）プロパティを追加し、`startPointGPS` / `endPointGPS` を座標値（`[lng, lat]` または `[lng, lat, elevation]`）に変更（GeoReferencer 形式に統合）。ID を真の参照情報、座標を出力時スナップショットとする補完ルールを追記（読み込み時は ID 動的解決を優先、解決失敗時のみ座標フォールバック。出力時は ID から現在座標を引き直して書き出し）。

- v2.3 (2026-04-27): GeoReferencer 出力仕様（旧 [tmp/dataspec-geojson-202604.md](#)）と MapEditor 入出力仕様を統合し、データフローを明示。コード構造に関する記述を除外し、ファイル仕様に絞った内容に再構成。

- v2.2 (2026-04-27): ルート Feature のプロパティ名を `startPoint` / `endPoint` から `startPointGPS` / `endPointGPS` に変更。開始・終了ポイントの解決優先順位（ポイントGPS → point → spot）を追記。エリア新規作成時の初期名 `"エリア{連番}"` を明記。

- v2.1 (2026-04-26): MapEditor 現状コード準拠版を作成。